

# SYSTEM PROMPT: Build a White-Label Multi-Tenant Gaming SaaS Platform

Build a production-grade white-label gaming platform with the following architecture and requirements.

## Objective

Create a centralized SaaS platform where a Super Admin can create and manage multiple gaming products.

Initial products:

- Dollara
- Product B
- Product C

All products must use:

- Same Django Backend
- Same Next.js Web Application
- Same React Native Mobile Application
- Same MySQL architecture

Only branding, domains, assets, and configuration should differ between products.

The platform must support onboarding unlimited future products without requiring code changes.

---

## Technology Stack

Backend:

- Python Django
- Django REST Framework
- MySQL
- Redis
- Celery

- JWT Authentication

Frontend:

- Next.js (App Router)
- TypeScript
- Redux Toolkit
- Tailwind CSS

Mobile:

- React Native
- Redux Toolkit
- React Navigation

Infrastructure:

- Nginx
  - Redis
  - MySQL
- 

## Multi-Tenant Architecture

Implement database-per-tenant architecture.

Structure:

Super Admin

→ Master MySQL Database

Products:

→ Dollara Database

→ Product B Database

→ Product C Database

Future products must be added dynamically.

No product data should ever be shared between databases.

Every product database is completely isolated.

---

# Tenant Resolution

The system must determine the tenant from:

- Domain
- Subdomain
- Mobile App Configuration

Examples:

dollara.com

→ connect to dollara\_db

productb.com

→ connect to productb\_db

productc.com

→ connect to productc\_db

Create a Tenant Resolver Service in Django that automatically switches database connections at runtime.

No hardcoded product logic.

---

# Super Admin Portal

Create a centralized Super Admin Dashboard.

Super Admin capabilities:

- Create Product
- Edit Product
- Disable Product
- Delete Product
- Create Tenant Database
- Manage Domains
- Manage Branding
- Manage Subscriptions
- Manage Licenses
- Manage Users
- View Revenue Analytics

- View Product Analytics
- Monitor Activity Logs
- Send Notifications
- Manage Mobile Builds

The Super Admin database must remain completely separate from tenant databases.

---

## White-Label Branding Engine

Create a dynamic branding engine.

Each product should support:

- Product Name
- Logo
- Favicon
- Theme Color
- Secondary Color
- Splash Screen
- App Icon
- Support Email
- Support Phone
- Terms URL
- Privacy Policy URL

All branding should load dynamically from configuration.

No hardcoded names such as Dollara inside the application source code.

---

## Authentication System

Implement:

- JWT Authentication
- Access Token
- Refresh Token
- Role-Based Access Control

Roles:

- Super Admin
- Product Admin
- User

Features:

- Login
  - Logout
  - Register
  - Forgot Password
  - OTP Verification
  - Session Management
- 

## User Features

Users should have access to:

- Dashboard
  - Profile Management
  - Wallet
  - Transaction History
  - Game History
  - Notifications
  - Referral System
  - Leaderboard
  - Rewards
  - Support Center
- 

## Wallet Engine

Create a complete wallet system.

Features:

- Deposit
- Withdraw
- Wallet Balance
- Transaction History

- Ledger Tracking
- Bonus Credits
- Referral Earnings

Every balance change must create a transaction record.

The system must be designed for high transaction volume.

---

## Game Engine Module

Create a modular game engine architecture.

Features:

- Game Catalog
- Game Categories
- Game Sessions
- Game History
- Win/Loss Tracking
- Statistics
- Leaderboards

Games must be pluggable modules.

New games should be installable without modifying existing code.

---

## Referral System

Features:

- Referral Codes
- Referral Links
- Multi-Level Referrals
- Referral Rewards
- Referral Analytics

Must be configurable per product.

---

# Notification System

Create a centralized notification engine.

Channels:

- In-App
- Email
- Push Notifications
- SMS (future-ready)

Features:

- Broadcast Notifications
  - User Notifications
  - Scheduled Notifications
- 

# Analytics Dashboard

Provide analytics for:

- Active Users
- New Registrations
- Revenue
- Deposits
- Withdrawals
- Transactions
- Game Activity
- Retention Metrics

Super Admin should see all products.

Product Admin should only see their product.

---

# Next.js Application

Build a modern responsive web application.

Modules:

- Authentication
- Dashboard
- Wallet
- Games
- Referrals
- Leaderboard
- Profile
- Notifications
- Admin Panel

Requirements:

- TypeScript
  - Redux Toolkit
  - API Layer
  - Responsive Design
  - Dark Mode Support
  - Dynamic Branding Support
- 

# React Native Application

Build a production-ready mobile application.

Features:

- Android
- iOS
- Dynamic Branding
- Push Notifications
- Wallet
- Games
- Profile
- Referrals
- Leaderboard

The same codebase must generate:

- Dollara App
- Product B App
- Product C App



through environment configuration only.

---

## API Design

Follow REST API standards.

Requirements:

- Versioned APIs
- Pagination
- Filtering
- Sorting
- Rate Limiting
- Validation
- Audit Logging
- Error Handling

Structure:

/api/v1/

Organize endpoints by module.

---

## Folder Structure

Backend:

- core
- tenants
- users
- wallets
- transactions
- games
- referrals
- notifications
- analytics
- admin
- services

- middleware

Frontend:

- app
- components
- modules
- services
- store
- hooks
- layouts

Mobile:

- src
  - screens
  - navigation
  - components
  - services
  - redux
  - hooks
- 

## Security Requirements

Implement:

- JWT
  - Password Hashing
  - CSRF Protection
  - Rate Limiting
  - Input Validation
  - SQL Injection Protection
  - XSS Protection
  - Secure Headers
  - Audit Logging
- 

## Scalability Requirements

The architecture must support:

- 100+ products
- Millions of users
- Horizontal API scaling
- Redis caching
- Queue processing
- Load balancing

No architectural shortcuts.

Design as an enterprise-grade SaaS platform.

---

## Deliverables

Generate:

1. Complete system architecture
2. Backend architecture
3. Frontend architecture
4. Mobile architecture
5. Folder structures
6. API specifications
7. Database connection strategy
8. Tenant resolution implementation
9. Authentication flow
10. Deployment architecture
11. Development roadmap
12. Production roadmap
13. Best practices
14. Source code implementation with clean architecture principles

The output should be production-ready, scalable, maintainable, and follow industry-standard SaaS architecture patterns.

# WHITE-LABEL MULTI-TENANT GAMING PLATFORM ARCHITECTURE

One Codebase – Multiple Products – Complete Isolation – Infinite Scale

